

HOI Business Center Developer Report

Prepared for: HOI Business Center

Prepared on: 20 May 2026

Guideline source: HOI_Business_Center_Developer_Guidelines.pdf

Original reference repository: https://github.com/Jatin-WD/hoi-business-center

Current repository: https://github.com/Jatin-WD/hoi-business-center-final

Current reported commit: 92c5e99 fix: restore full admin dashboard

Executive Summary

The current project is a full-stack HOI Business Center website with React/TypeScript frontend, Node.js/Express backend, PostgreSQL/Supabase database support, seeded catalog data, authentication, customer submission flows, and a restored admin dashboard. The website is ready for GitHub-based deployment after production environment variables are configured.

Guideline Item	Status	Evidence / Note
Frontend language	Aligned	React 18 + TypeScript + Vite.
Backend language	Aligned	Node.js 22 + Express REST API.
Database	Aligned	PostgreSQL/Supabase via DATABASE_URL and pg .
Data stored in database	Aligned	Services, packages, venues, events, users, inquiries, manpower requests, bookings, CMS content, admin replies and notifications are DB-backed.
Frontend components below 200 lines	Aligned	No src/pages or src/components TSX component exceeds 200 lines. Largest admin page is 199 lines.
Total source LOC below 6,000	Not currently aligned	Current source LOC is 8,056 after restoring the full admin dashboard. This is the main remaining guideline deviation.
No file above 200 lines	Partially aligned	Frontend components comply. Backend backend/routes/admin.js is 624 lines and data seed file is 550 lines.
CI/CD	Configured	.github/workflows/ci.yml runs lint/build on push and pull requests.

Technology Stack

Layer	Technology
-------	------------

Frontend	React 18, TypeScript, Vite, Wouter, TanStack React Query
Styling	Tailwind CSS, Lucide React icons, HOI brand palette
Backend	Node.js 22, Express, JavaScript ES modules
Database	PostgreSQL/Supabase, initialized through <code>backend/config/schema.js</code>
Authentication	JWT, bcrypt password hashing, admin token guard
Email	Nodemailer for requirement notifications
Deployment Target	GitHub deployment to Hostinger Node.js app or similar Node hosting

Stage 1 - Code Cleanup and Structure

Cleanup Completed

- Removed hardcoded frontend catalog dependency from service pages and moved primary catalog records into database seed data.
- Removed MySQL deployment path from active configuration and standardized production config around PostgreSQL/Supabase.
- Removed unsafe JWT fallback in production; `JWT_SECRET` is required.
- Protected sensitive admin-only submission endpoints through admin middleware.
- Restored admin dashboard into split frontend components so UI component files remain under 200 lines.

Current Line Count

Metric	Result
Total source LOC excluding <code>node_modules</code> and <code>dist</code>	8,056
Frontend component files above 200 lines	0
Backend/source files above 200 lines	<code>backend/routes/admin.js</code> (624), <code>backend/data/seed-data.js</code> (550)

Note: The admin dashboard requirement increased the total LOC. To fully satisfy the 6,000 LOC guideline, the next cleanup should split `backend/routes/admin.js` into smaller route modules and reduce or externalize seed data.

Stage 2 - Backend API and Database

Implemented API Endpoints

Endpoint	Method	Status	Purpose
/api/auth/register	POST	Implemented	User registration
/api/auth/register/verify	POST	Implemented	Email verification
/api/auth/login	POST	Implemented	User login
/api/auth/logout	POST	Implemented	User logout
/api/auth/me	GET	Implemented	Session profile
/api/services	GET	Implemented	Service menu/catalog
/api/packages	GET	Implemented	Package catalog
/api/venues	GET	Implemented	Venue catalog
/api/events	GET	Implemented	Event calendar
/api/inquiries	POST	Implemented	Service requirement form
/api/manpower	POST	Implemented	Manpower application form
/api/bookings	POST/GET	Implemented	Authenticated bookings
/api/admin/login	POST	Implemented	Admin login
/api/admin/dashboard	GET	Implemented	Admin dashboard data
/api/admin/resource/:resource	GET/POST/PUT/DELETE	Implemented	Admin CMS/resource management
/api/admin/submission/:source/:id	PATCH/DELETE/POST	Implemented	Submission status, delete and reply workflow
/api/cms/content	GET/POST/DELETE	Implemented	Website CMS content

Database Tables

Table	Purpose
users	Website users and admin users
venues	Venue and location records
services	Service categories and menu links
packages	Package pages, price text, inclusions and exclusions

events	Event calendar records
inquiries	Contact and service requirements
manpower_requests	Manpower applications and uploaded document references
bookings	Customer booking requests
cms_content	Editable website text/theme/admin CMS content
admin_replies	Admin replies to user submissions
notification_dismissals	Admin notification dismissal state

Database Seed Data

Seed data is stored in `backend/data/seed-data.js` and loaded by `backend/scripts/init-db.js`. On startup, `seedDatabaseIfEmpty()` initializes schema and inserts catalog data when required.

Stage 3 - Frontend Integration

Feature	Status	Notes
Home/service browsing	Working	Reads services, packages and venues from backend API.
Signup/login	Working	Uses JWT auth and backend validation.
Contact inquiry	Working	Saves requirement data to database and can notify email.
Manpower application	Working	Supports role-specific fields and upload references.
Bookings	Working	Authenticated booking API is implemented.
Admin login	Working	Available at <code>/admin-login</code> .
Admin dashboard	Working	Available at <code>/admin</code> with requirements, notifications, users, admins, CMS content and resource management.
404/error states	Working	Custom not-found page and API error handling are present.

Stage 4 - Quality, Testing and Deployment

Checkpoint	Status	Evidence
Frontend lint	Passed	<code>pnpm run lint</code>
Production build	Passed	<code>pnpm run build</code>
Backend syntax check	Passed	<code>node --check backend/routes/admin.js</code> and <code>node --check backend/routes/cms.js</code>
Frontend component size	Passed	No frontend component file above 200 lines.
CI/CD	Configured	GitHub Actions workflow exists in <code>.github/workflows/ci.yml</code> .
Formal unit test coverage	Not installed	Project currently relies on lint/build and API smoke readiness. A formal coverage tool is still recommended.

Deployment Requirements

Set these production environment variables before deployment:

- `NODE_ENV=production`
- `PORT=3000` or host-provided port
- `FRONTEND_URL=https://your-domain.com`
- `JWT_SECRET` with a strong random value
- `ADMIN_EMAIL` and `ADMIN_PASSWORD`
- `DB_CLIENT=postgres`
- `DATABASE_URL=postgresql://...` from Supabase/production PostgreSQL
- `DB_SSL=true` for Supabase
- `REQUIREMENT_EMAIL` , `EMAIL_USER` , `EMAIL_PASS` for emails

Final Notes

- The project is functionally deployable after environment variables are configured correctly.
- The restored admin dashboard meets the user's requested frontend component limit.
- The remaining guideline improvement is backend size cleanup: split `backend/routes/admin.js` and reduce total LOC below 6,000 if strict PM review requires it.